PEC 1

Pau Pérez Rebassa

Url del reporsitorio de *github*

Enlace al repositorio de github: https://github.com/Palitop/Perez-Rebassa-Pau-PEC1

Preparación de la PEC

Lo primero de todo, se va a ejecutar el siguiente código para poder evitar las salidas no deseadas de algunos comandos.

```
paquetes = c("GenomicRanges", "Biobase", "GenomeInfoDb", "matrixStats", "MatrixGenerics",
             "stats4", "BiocGenerics", "S4Vectors", "SummarizedExperiment")
suppressMessages(suppressWarnings(invisible(lapply(paquetes, library,
                                                   character.only = TRUE))))
```

Descarga del *dataset*

Para realizar esta práctica he escogido el github que se nos ha facilitado en el enuciado con dirección url de **https://github.com/nutrimetabolomics/metaboData/** y de todos los *datasets* que hay almacenado he escogido el de la cachexia, porque me parece un tema bastante interesante y poco conocido entre la población.

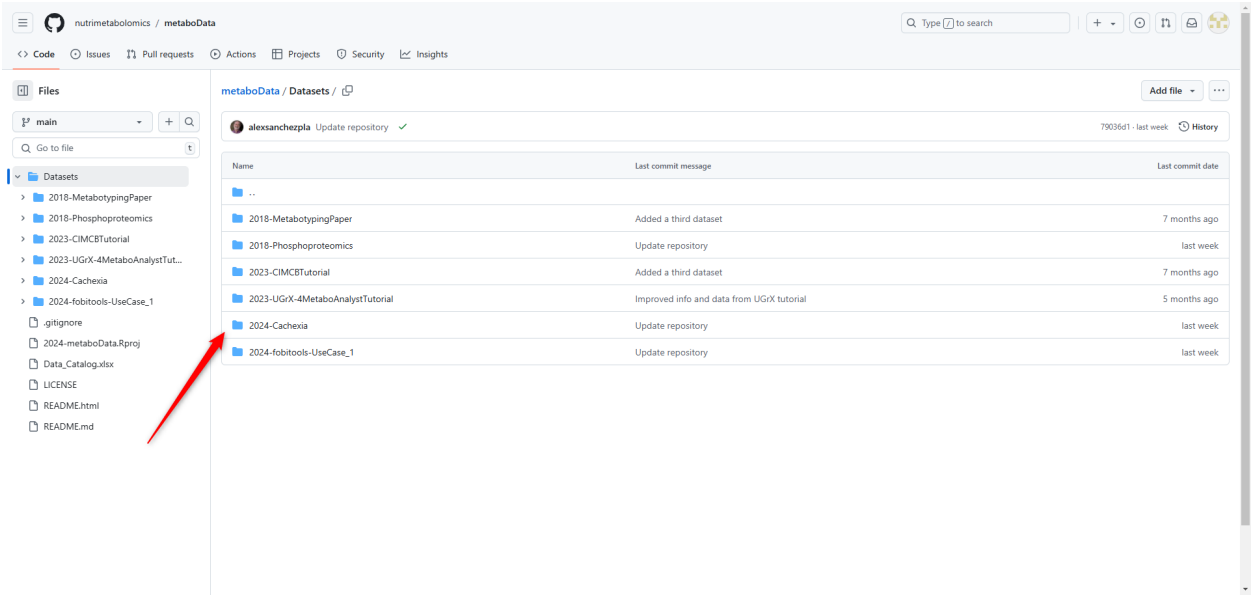


Figure 1: Captura del github proporacionado en el enunciado

Crear el *SummarizedExperiment*

Leemos el fichero descargado del *github* del enunciado de la práctica y lo cargamos en una variable.

```
cachexia = read.csv("human_cachexia.csv")
```

Eliminamos las dos primeras columnas que identifican el sujeto del experimento y su condición para poder obtener los datos en bruto del *dataset*.

```
datos_bruto = cachexia[, c(-1, -2)]
```

Convertimos los datos en bruto en una matriz para poder crear próximamente el objeto *SummerizedExperiment* y la transponemos para que cuadren el número de columnas y de filas.

```
datos_bruto_matriz = t(as.matrix(datos_bruto))
```

Preparamos los datos de la columna.

```
column_Data = data.frame(
  PatientID = cachexia[, 1],
  MuscleLoss = cachexia[, 2],
  row.names = cachexia[, 1],
  stringsAsFactors = FALSE
)
```

Preparamos la metadata.

```
metadata = data.frame(
  VarNames = colnames(datos_bruto),
  TypeCol = sapply(datos_bruto, class)
)
```

Finalmente, creamos el objeto *SummarizedExperiment* con los datos que hemos preparado antes con el *dataset* elegido.

```
se = SummarizedExperiment(
    assays = list(counts = datos_bruto_matriz),
    colData = column_Data,
    metadata = metadata
)
```

Exploración del *dataset*

Como inicio de la exploración del dataset, podemos ver un breve resumen del objeto.

```
se
```

```
## class: SummarizedExperiment
## dim: 63 77
## metadata(2): VarNames TypeCol
## assays(1): counts
```

```
## rownames(63): X1.6.Anhydro.beta.D.glucose X1.Methylnicotinamide ...
##   pi.Methylhistidine tau.Methylhistidine
## rowData names(0):
## colnames(77): PIF_178 PIF_087 ... NETL_003_V1 NETL_003_V2
## colData names(2): PatientID MuscleLoss
```

Como podemos ver en el resultado del resumen, la **dimensión** de los datos es de 63 variables y 77 registros. La metadata consiste en saber cual es el nombre de las variables y el tipo de dato que contienen. Las variables que identifican, en este caso, al paciente, son **PatientID**, el id del paciente y **MuscleLoss** con los valores de *cachexia* y *control*.

Por otro lado, podemos ver si los datos en bruto contienen algún valor *na* en los datos.

```r
colSums(is.na(assay(se, "counts")))
```

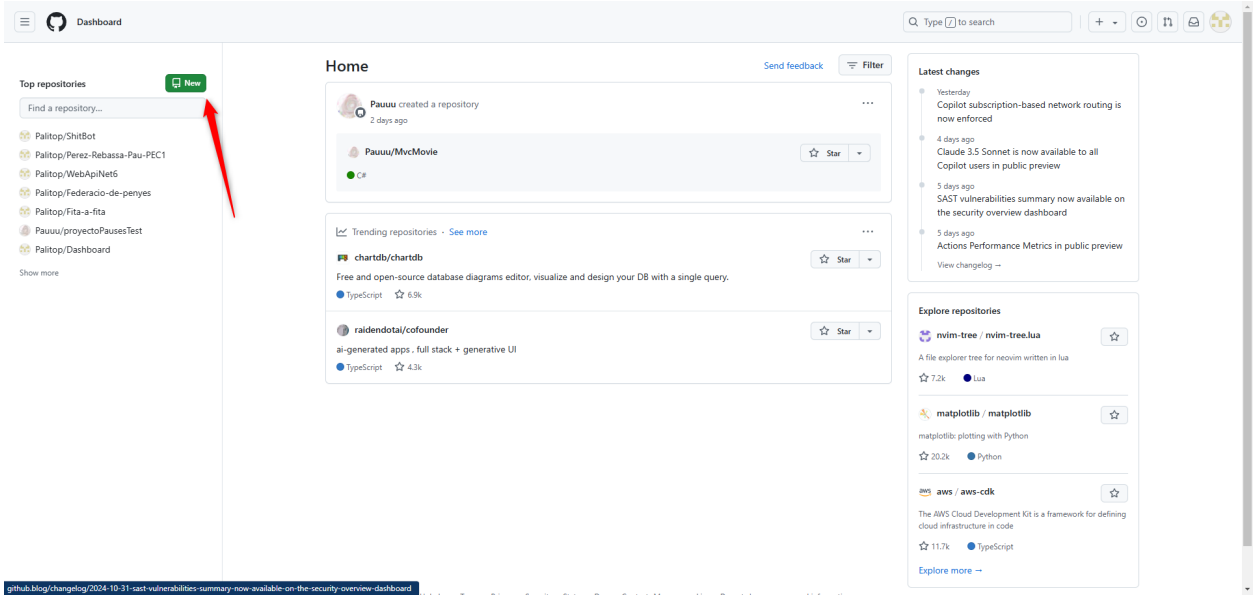
```
##      PIF_178      PIF_087      PIF_090  NETL_005_V1      PIF_115      PIF_110
##            0            0            0            0            0            0
##  NETL_019_V1 NETCR_014_V1 NETCR_014_V2      PIF_154  NETL_022_V1  NETL_022_V2
##            0            0            0            0            0            0
##  NETL_008_V1      PIF_146      PIF_119      PIF_099      PIF_162      PIF_160
##            0            0            0            0            0            0
##      PIF_113      PIF_143 NETCR_007_V1 NETCR_007_V2      PIF_137      PIF_100
##            0            0            0            0            0            0
##  NETL_004_V1      PIF_094      PIF_132      PIF_163 NETCR_003_V1  NETL_028_V1
##            0            0            0            0            0            0
##  NETL_028_V2 NETCR_013_V1  NETL_020_V1  NETL_020_V2      PIF_192 NETCR_012_V1
##            0            0            0            0            0            0
## NETCR_012_V2      PIF_089 NETCR_002_V1      PIF_179      PIF_114 NETCR_006_V1
##            0            0            0            0            0            0
##      PIF_141 NETCR_025_V1 NETCR_025_V2 NETCR_016_V1      PIF_116      PIF_191
##            0            0            0            0            0            0
##      PIF_164  NETL_013_V1      PIF_188      PIF_195 NETCR_015_V1      PIF_102
##            0            0            0            0            0            0
##  NETL_010_V1  NETL_010_V2  NETL_001_V1 NETCR_015_V2 NETCR_005_V1      PIF_111
##            0            0            0            0            0            0
##      PIF_171 NETCR_008_V1 NETCR_008_V2  NETL_017_V1  NETL_017_V2  NETL_002_V1
##            0            0            0            0            0            0
##  NETL_002_V2      PIF_190 NETCR_009_V1 NETCR_009_V2  NETL_007_V1      PIF_112
##            0            0            0            0            0            0
## NETCR_019_V2  NETL_012_V1  NETL_012_V2  NETL_003_V1  NETL_003_V2
##            0            0            0            0            0
```

Los resultados nos dicen que no hay ningún registro con valores nulos.

Subida de los datos a repositorio nuevo

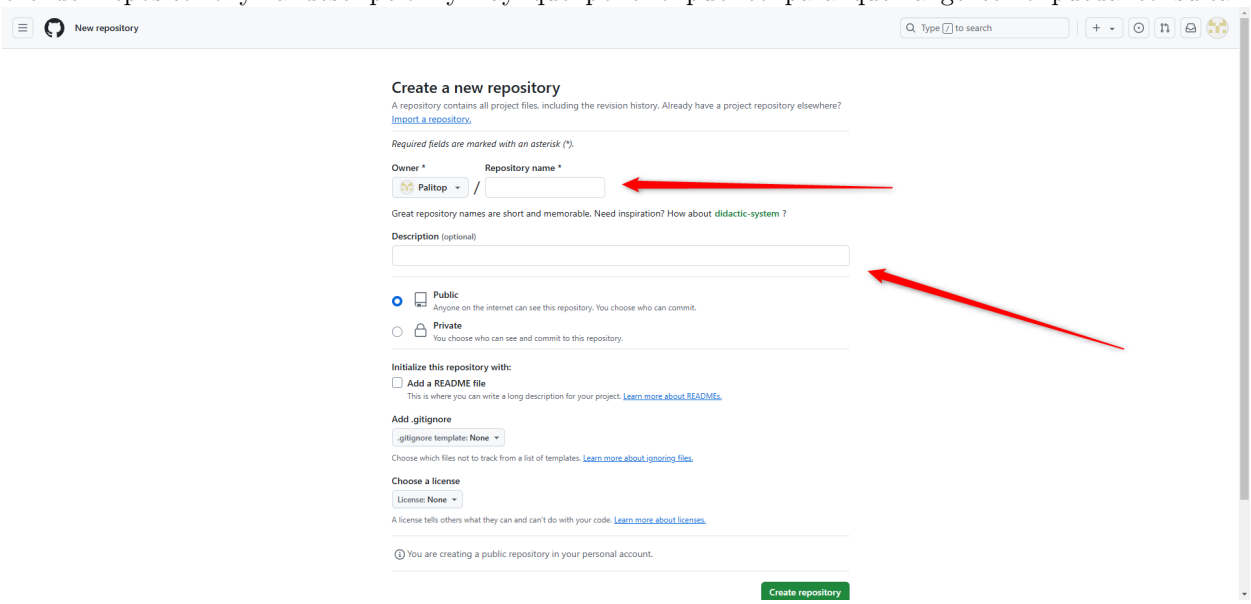
En mi caso, yo ya tenía una cuenta en la plataforma de reporsitorios *github*, por lo tanto, me he saltado el paso de crear la cuenta.

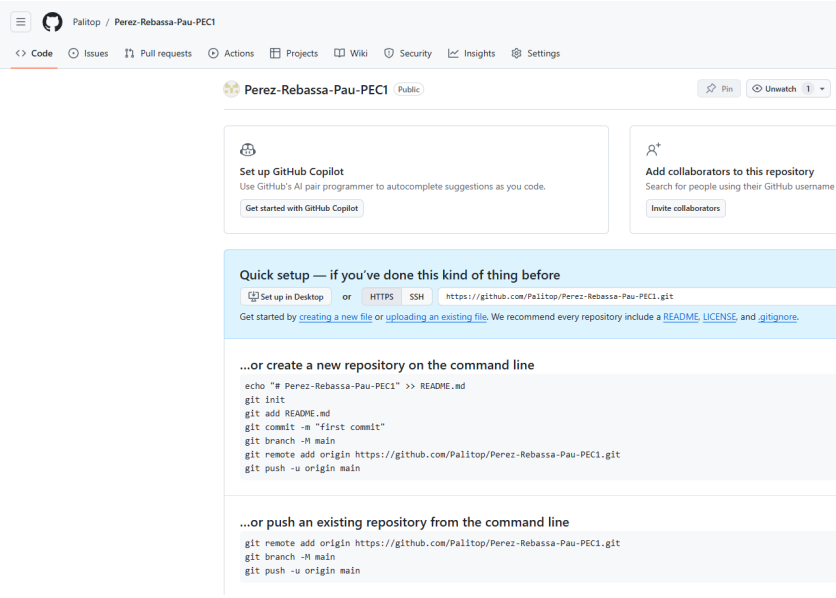
Hay que apretar el botón que se marca en la siguiente captura para poder crear un nuevo repositorio en el per-



fil. github.blog/changelog/2024-10-31-sast-vulnerabilities-summary-now-available-on-the-security-overview-dashboard

Nos saldrá la siguiente pantalla donde tendremos que rellenar los datos requeridos, como el nombre del repositorio y la descripción y hay que ponerlo público para que la gente lo pueda consultar.





Con todo esto obtenemos el repositorio creado vacío.